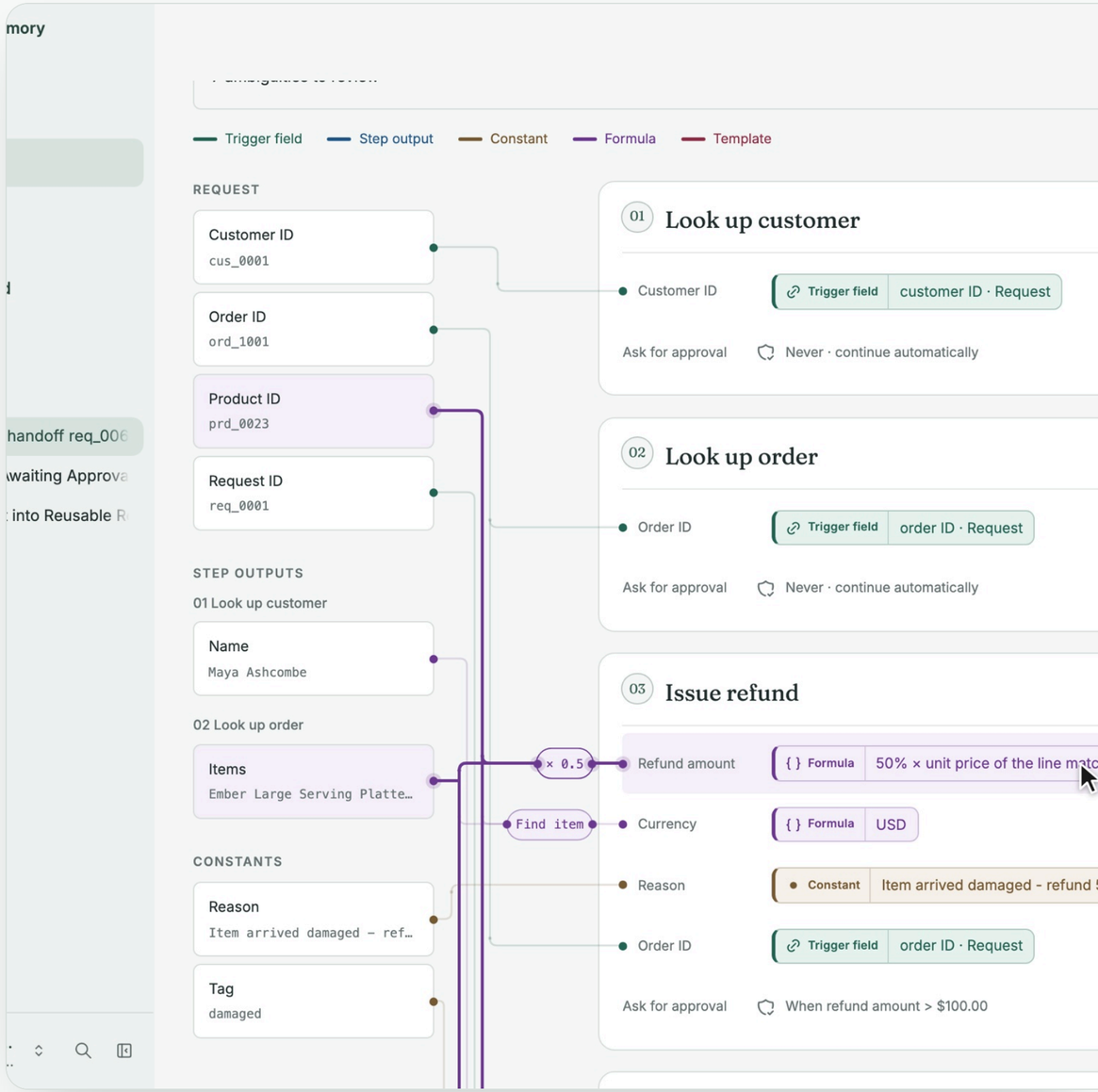
AGENT-NATIVE HACKATHON 2026

Muscle Memory

Show it once.
Your app remembers.

A support inbox that learns a routine from one demonstration, then handles matching requests with **`0 LLM tokens`**.

Evgeny Vinokur · demo shop: Kiln & Co. (synthetic data)



THE PROBLEM

Support work repeats. Agents re-reason it every time.

- Most support email is the same few questions: **broken item, where's my order, returns, address changes**.
- An AI agent can answer them, but **every answer is a full model run**, spent on decisions the operator has already made.
- In our recorded session, one agent-handled request cost `101K tokens · 6.8 s`.

HOW IT WORKS

Show it once. Then it runs without the model.

01

Teach

Operator

The operator handles one request in the normal UI. Every click is a typed action, recorded with its real inputs and outputs.

02

Draft

Code · 0 tokens

Code wires every argument to its source. Anything it can't prove becomes a question.

03

Crystallize

Agent

The agent names the recipe, settles the open choices and dry-runs it on another customer.

04

Guard

Operator

An approval rule, a backtest on resolved requests, then an explicit activation.

05

Remember

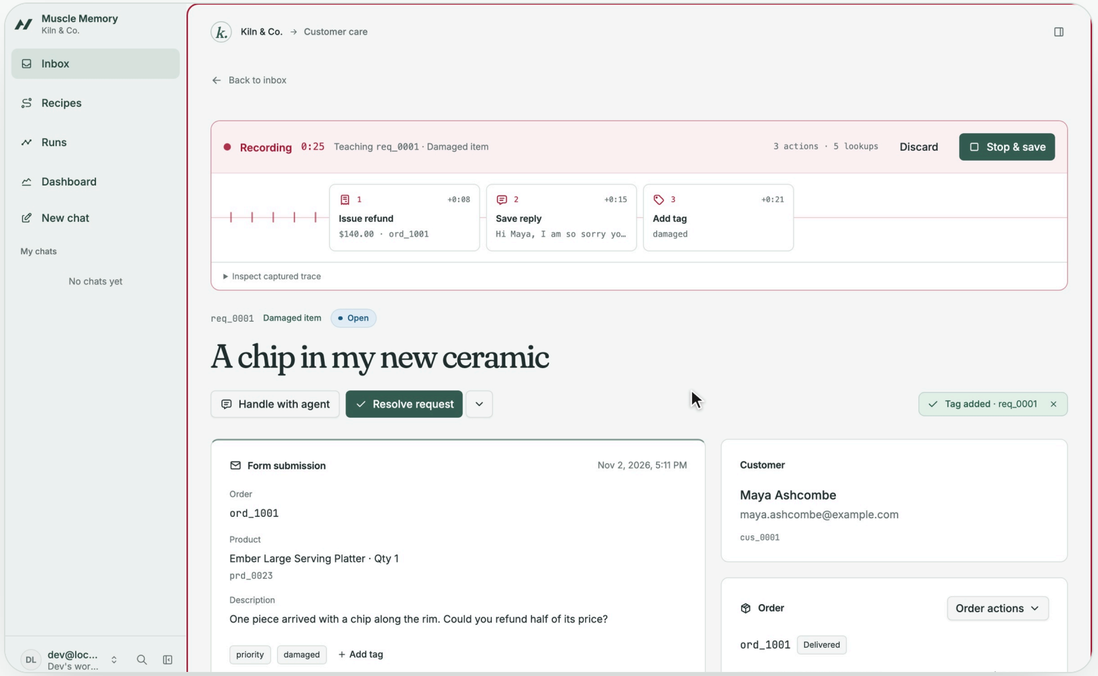
Code · 0 tokens

Matching requests run deterministically. Big refunds wait for approval; unfamiliar work goes to the agent.

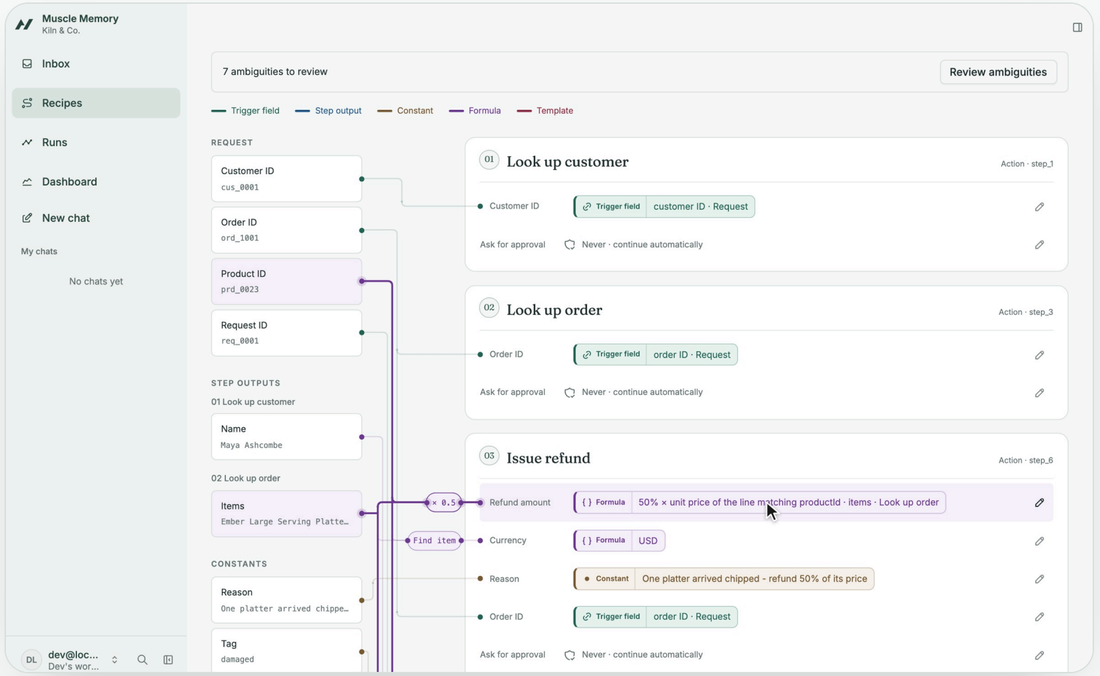
Corrections become proposed recipe versions, and an agent's resolved run can be crystallized too.

TEACH → DRAFT

A recording of actions, not pixels.

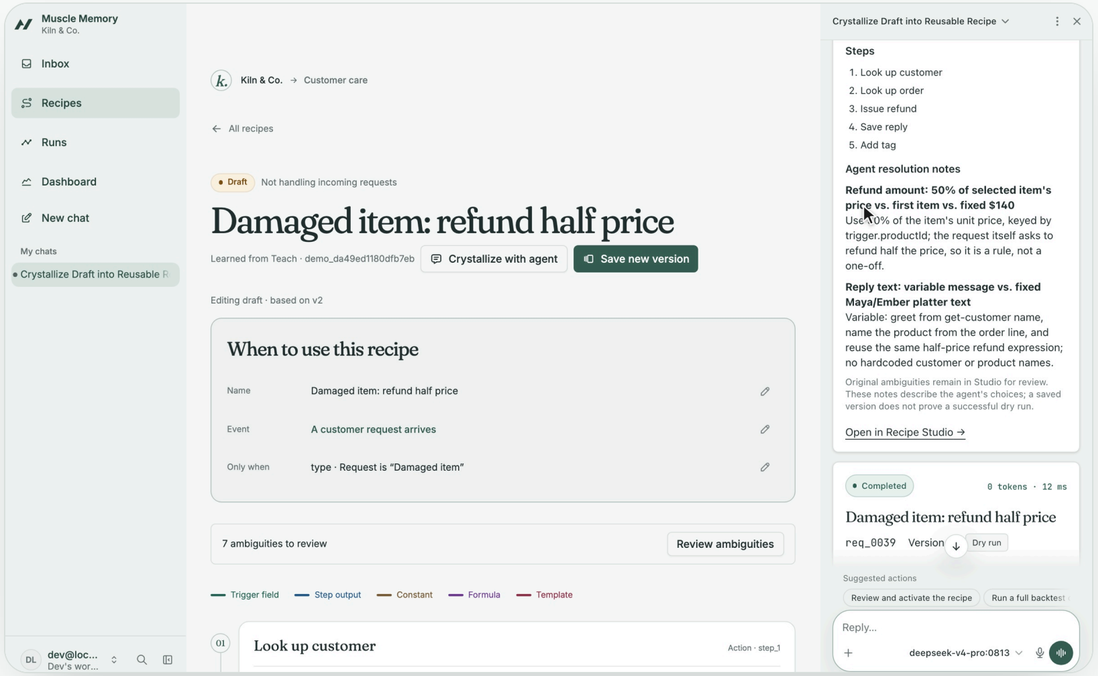


Teach. Refund, reply and tag land as frames on a filmstrip, each an exact typed action.

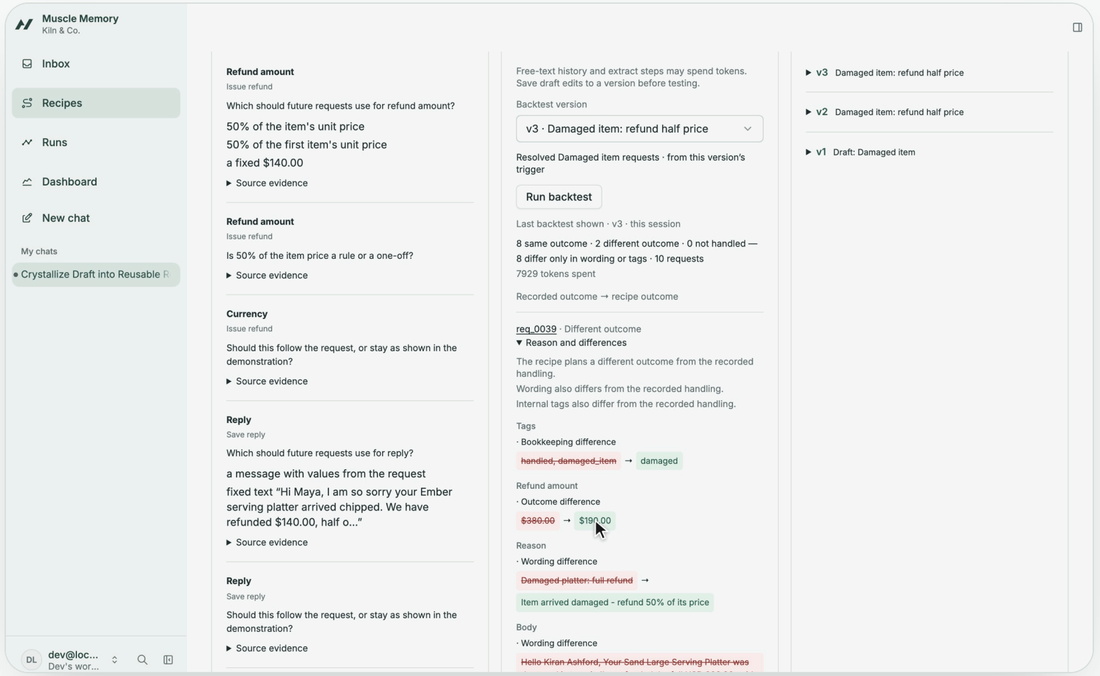


Recipe Studio. Every argument is wired to the request, an earlier lookup, a constant or a formula like `× 0.5`.

The agent does what code can't. You keep the final say.



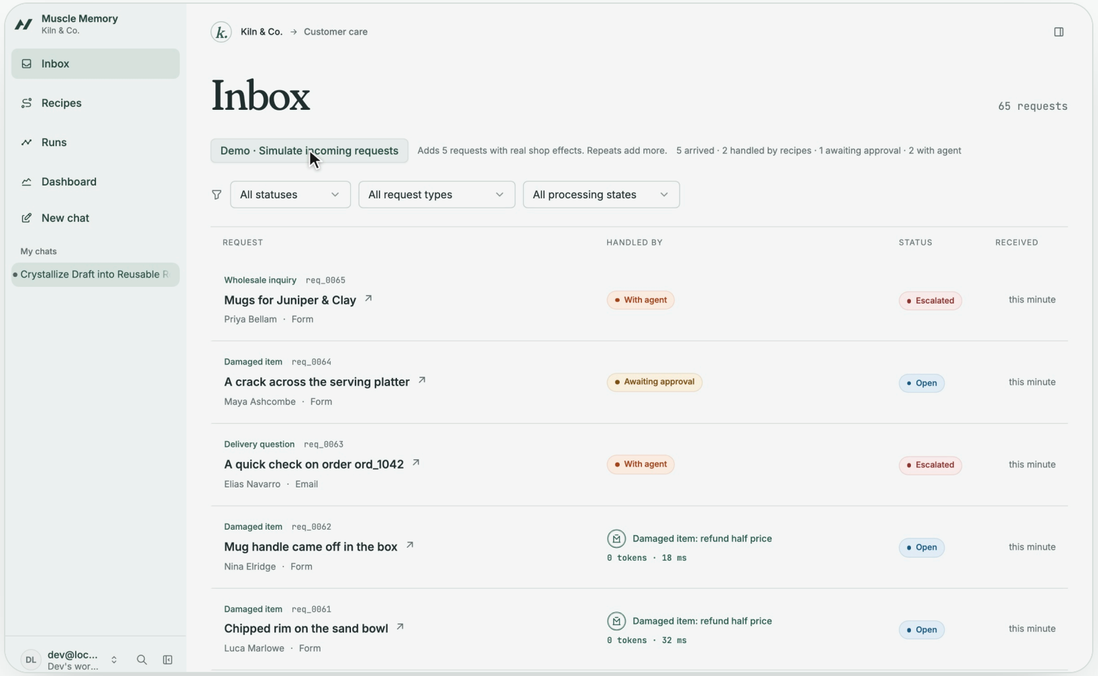
Crystallize with agent. It names the recipe, resolves each open choice with a note and proves it on another customer at `0 tokens`.



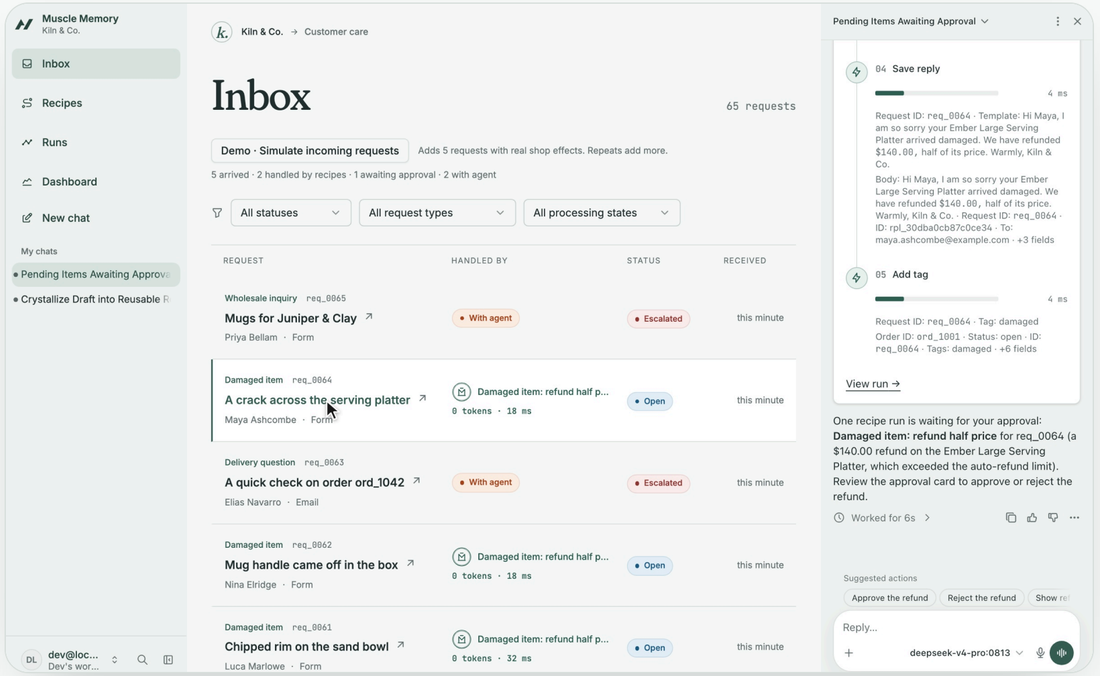
Approval rule and backtest. Refunds over $100 need approval. Every difference from past handling comes with its reason: `$380.00 → $190.00`.

REMEMBER

New requests now run without the model.



Handled by the recipe: `0 tokens · 18–32 ms`. The big refund waits for approval; the wholesale quote goes to the agent.



Approve in chat. Ask the agent what needs you, approve on its run card, and the inbox updates at once. It is the same action as the UI button.

MEASURED, NOT ESTIMATED

Less model. More memory.

Agent per request

101K tokens

6.8 s

Recipe per request

0 tokens

22.7 ms

302× faster

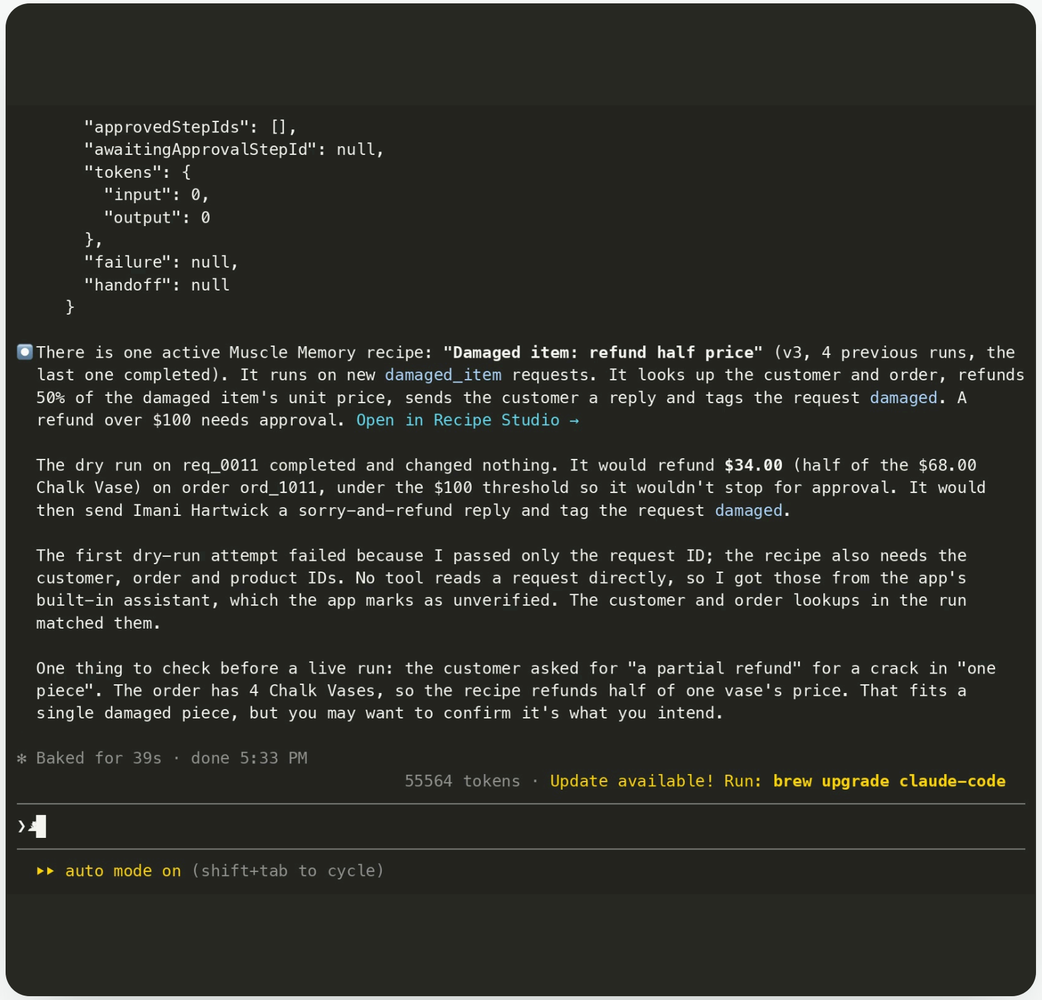
101K tokens avoided per request

Recorded session on 2026-11-02, live model deepseek-v4-pro via Ollama Cloud. Agent n = 1 and recipe n = 3, on different requests. The dashboard reads every number from run and usage records; there are no fixed savings.

WHY AGENT-NATIVE

One set of actions for the UI, the agent, MCP and the CLI.

- **A click is an executable step.** The UI and the agent call the same typed actions, so a human demonstration can be replayed as code.
- **A registry without a deploy.** Activated recipes are published as runtime skills, and MCP and CLI clients list and run them.
- **The chat is part of the workflow.** Run, approval and proposal cards render in the agent chat, and the agent sees the current screen.



Claude Code over MCP: it lists the active recipe and dry-runs it on another request, with 0 tokens and no changes made.

ARCHITECTURE

Clean Architecture in one pnpm workspace.

`apps/web`

Agent-Native app

Actions are thin controllers shared by the UI, agent, MCP and CLI. Adapters for PGlite/Drizzle, the action registry, the event bus and skills. React UI.

`packages/recipe-engine`

Recipe engine

Recipe DSL, provenance inference, deterministic executor, approvals, backtests. Pure TypeScript with no knowledge of the shop.

`packages/shop-domain`

Shop domain

Orders, refunds, replies and policies as pure use-cases with ports.

- Dependencies point inward; `dependency-cruiser` enforces the rule.
- `1,196` unit and integration tests, architecture tests and a Playwright end-to-end flow. Strict TypeScript.

HONEST SCOPE

What is real, and what was reused.

- **Real app, synthetic shop.** All customers and orders are synthetic. Replies are stored, not emailed. The app runs locally.
- **Reused:** the Agent-Native chat template (`@agent-native/core 0.222.2`) and open-source libraries. Everything app-specific was built during the hackathon.
- **AI tools:** OpenAI Codex CLI (gpt-6.1-sol) wrote the code from task work orders; Claude Code (Opus 5.5) planned, reviewed and verified each change. In-app models run on Ollama Cloud (deepseek-v4-pro, glm-5.3-flash). The demo narration is synthetic.

Demo (3:47)
`clips.agent-native.com/share/IgzNxnFLER7y`

Code
`github.com/Wertyk/muscle-memory`